



High-level implementable methods for automated building code compliance checking

Jin-Kook Lee^a, Kyunghyun Cho^b, Hyeokjin Choi^b, Soohyung Choi^a, Sumin Kim^c,
Seung Hyun Cha^{b,*}

^a Dept of Interior Architecture and Built Environment, Yonsei University, Seoul, Republic of Korea

^b Graduate School of Culture Technology, Korea Advanced Institute of Science and Technology, Daejeon, Republic of Korea

^c Dept of Architecture and Architectural Engineering, Yonsei University, Seoul, Republic of Korea

ARTICLE INFO

Keywords:

High-level method
Implementable method
Automated building code compliance checking
Building permit
Building information modeling (BIM)

ABSTRACT

This paper presents an approach for defining high-level implementable methods to improve their low-level rule-checking procedures. This is part of an effort to develop challenging building information modeling (BIM)-enabled applications for building permits through automated code compliance checking. The main approach described here aims to alleviate the time-consuming and error-prone tasks of translating natural language into explicitly defined rules. An explicit expression of design requirements is key to building projects involving collaboration between architects, code experts, and developers. To maximize the generalization, neutralization, and reusability of the given rules in natural (written) language, we propose a series of high-level implementable computer programming methods (operators); these can be beneficial for translating verb phrases in building act sentences with minimal ambiguity as well as representing the peculiar properties of building objects without conflicts. Lastly, we demonstrate its application in code compliance checking by employing KBimCode and the developed rule-checking software.

1. Introduction

National building acts regulate several design requirements, including building permits, and usually have legally binding force. Architects and government agencies typically review the compliance of building designs with these acts. In the building planning phase, building permits must be obtained from an administrative agency. Code compliance checking is usually a time-consuming and error-prone task; however, it is important to prevent unnecessary legal disputes. Building information modeling (BIM) has increased its influence in the architecture, engineering, construction, and facility management (AEC-FM) industry (Lee et al., 2018a; Choi et al., 2015; Koo and Shin, 2018); thus, many research projects are currently being conducted to automate the code compliance checking process for various BIM applications (Amor and Dimyadi, 2021; Luo et al., 2022). These projects are led by several different parties, including governments (Lee et al., 2012; Eastman et al., 2008; Han et al., 1997; Greenwood et al., 2010; Lee et al., 2012; Raslan and Davies, 2006). The South Korean administration has also introduced e-submission systems as part of the development of an overall electronic

administration service (Seumter, 2022). This national-level interest in BIM-based automated code compliance checking has resulted in the creation of international groups of government representatives, such as the BuildingSMART Regulatory Room (Regulatory Room, 2022).

Several international organizations have attempted to enhance the productivity, performance, and overall quality of building design using BIM for automated design checking against national building permit requirements (Kim, 2015). Because most building acts are written in natural language, their building-permit-related requirements should be formalized and encoded into a computer-readable format that enables automated reasoning. Therefore, extensive studies have been conducted to transform natural written language into computable formats. Many natural language processing (NLP) methods have been explored, including first-order logic (FOL), context-free grammar (CFG), and context-sensitive grammar (Hutchins, 2005; Barwise, 1977; Hopcroft and Ullman, 1979; Linz, 2011). In the field of AEC-FM, attempts have been made to translate the requirements written in design guidelines, requests for proposals (RFPs), and building codes into computable formats for automated compliance checking. Related research activities

* Corresponding author.

E-mail address: shcha@kaist.ac.kr (S.H. Cha).

<https://doi.org/10.1016/j.dibe.2023.100174>

Received 14 February 2023; Received in revised form 29 April 2023; Accepted 12 May 2023

Available online 24 May 2023

2666-1659/© 2023 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

date back to the early 1960s, when developers utilized a decision table to formalize the specifications of the American Institute of Steel Construction (Solihin and Eastman, 2015).

Various analysis methods have been adopted in recent studies, some of which are still ongoing. In Singapore, the Construction and Real Estate Network (CORENET) project has hardcoded the national building code using a programming language (E-Info and CORENET Singapore, 2022; Teo and Cheng, 2006). The US General Services Administration Courthouse project has parameterized its circulation rules according to a predefined parametric table (Eastman et al., 2009; Lee, 2010); Lee and Eastman (Lee, 2012) developed a domain-specific language, named building environment and rule analysis (BERA), to analyze and define the building circulation and spatial programming rules. The Australian DesignCheck project has formalized and encoded the national building code using the EDM rule schema, based on the EXPRESS language (Ding et al., 2006; EDM: EXPRESS Data Manager, 2022; ISO TC184/SC4, 1997; Parr, 2008). Uhm et al. (2015) analyzed RFPs for large public buildings in Korea and formalized them using four morphemes: object, method, level of strictness, and others; these formalized rules were translated into the semantic web rule language (SWRL) and Python. Malsane et al. (2015) developed a semantically rich object model by expanding the Industry Foundation Classes (IFC) for compliance checking of the England and Wales Building Regulations. Zhang et al. developed algorithms that automatically extract information from the International Building Code (IBC) and transform the results into a formalized format using an NLP approach (Zhang and El-Gohary, 2013, 2015a). Lee et al. proposed an automated IFC evaluation process for model view definitions that are subsets of the IFC schema (Lee et al., 2015a, 2018b). Moreover, in a recent study on validation logic, they explicitly demonstrated the rule logic and mechanism of automated BIM data checking pertaining to data exchange standards (Lee et al., 2015b, 2019).

The aforementioned research demonstrates the possibility of various approaches to code compliance checking. However, the application of these methods to the translation of natural language into explicitly defined rules remains limited by the locality and distinct characteristics of natural languages. When planning a translation strategy for a building act, the following aspects of the act should be carefully considered.

- Complex relations: the sentences are commonly cross-referenced. In addition, the display of relationships is inconsistent (i.e., relations are marked randomly in referencing or referenced sentences). This increases complexity, which in turn increases the number of discrepancies in interpreted sentences.
- Integrity: the building act carries legally binding force. Its interpretation should ensure integrity without errors, which might otherwise lead to unnecessary conflict. Therefore, the integrity of the interpretation must be guaranteed, and the responsibility for possible errors must be clear.

Thus, to overcome the problems in the rule-making process, this study aimed to develop high-level methods suitable for a certain level of general design requirements. The expression “high-level implementable method” refers to an explicit intermediate pseudo-code definition of computer-executable rules useable in rule-checking software. This enables software developers to improve their low-level rule-checking procedures by including highly sophisticated geometric calculations. Because high-level methods are mostly responsible for representing verb phrases, they can be implemented via mapping with low-level libraries embedded in rule-checking software developed by non-experts in the complex issues of building codes.

2. Literature review

There have been several attempts to reduce manual processing and errors in ACCC, and the related literature can be categorized using four methodologies. Three types of approaches that do not use NLP (i.e.,

Object-based, Logical, and Semantic/Ontological approaches) and one type of approach using NLP. Object-based approaches are a type of computational method that involves creating and utilizing objects and rule objects to solve complex problems (Sacks et al., 2017). This approach relies on the adoption of machine-learning techniques and the use of various data features to achieve high-accuracy results. In this approach, objects are defined as entities that possess specific properties, and rule objects are sets of rules that govern the interactions between objects (Mazairac and Beetz, 2013). For example, Nguyen et al. (Nguyen and Kim, 2011) introduced a framework for a collaborative code compliance checking environment using BIM. In addition, Getuli et al. (2017) applied customized rulesets for code-checking, and Patlakas et al. (2018) studied the ACCC framework utilizing multi-dimensional data fitting (MDDF) for complex design. Furthermore, two studies applied the ACCC at a sub-task level: Luo et al. (Luo and Gong, 2015) proposed a BIM-based code checking system for deep foundation construction, utilizing decision tables and IDM-like processes, and Kince-lova et al. (2019) explored the potential of the ACCC for the fire safety of timber structures.

Logical approaches involve the use of formal logic and logical languages to translate building rules and regulations into a computable format that can then be incorporated into the BIM (Guo et al., 2021). This approach entails the interpretation of human language regulations and their conversion into logical languages, such as first-order predicate logic, and is utilized to enrich the BIM model with comprehensive information, resulting in an accurate representation of a building’s design and functionality (Nawari, 2013). Using logical approaches, complex building rules can be efficiently and effectively incorporated into the BIM, providing a systematic and structured method for this process (Nawari, 2012). As a more advanced method, attempts have been made to ensure IFC model view definition conformance through a rule-based modularized validation platform (Lee et al., 2015a), as well as to create and utilize a domain-specific language that embeds the syntax and scheme of building model objects and delivers them to model validation applications (Lee, 2012; Lee and Eastman, 2019).

Semantic/ontological approaches are computational methods that rely on ontology modeling to represent and define concepts and their relationships (Gruber, 1993). These approaches use classes and properties to capture the fundamental aspects of a domain, thus simplifying the representation of complex knowledge and data in a structured format. By employing ontology-based models, such approaches enable the development of comprehensive and accurate representations of intricate regulatory frameworks, and facilitate the automated checking of compliance against these regulations (Wang et al., 2011). Numerous studies in the construction industry have explored ontology-based approaches for diverse aspects, such as compliance checking. For example, construction quality has been evaluated through ontology-based semantic modeling of regulation constraints (Zhong et al., 2012). Xu et al. (Xu and Cai, 2020) suggested a semantic approach that integrates heterogeneous data and enables ACCC for underground utilities using logic and spatial reasoning. Zhong et al. (2018) introduced a framework that supports environmental monitoring and compliance checking in a BIM environment to address the challenges of information sharing and semantic interoperability. In addition, a systematic review of ontology research within the construction industry has been conducted (Zhong et al., 2019).

Although attempts have been made to automate code compliance checking, some limitations remain. Issues with hard-coded implementations make it difficult to apply these approaches to advanced systems. Even when using machine learning, additional human effort is essential to create datasets for model training and convert them into applications (Zhang and El-Gohary, 2017). NLP can potentially solve these problems. BERT, a pretrained text encoder, captures linguistic information within the network, representing the steps of the traditional NLP pipeline in an interpretable and localizable manner, enabling the model to dynamically adjust the pipeline based on higher-level

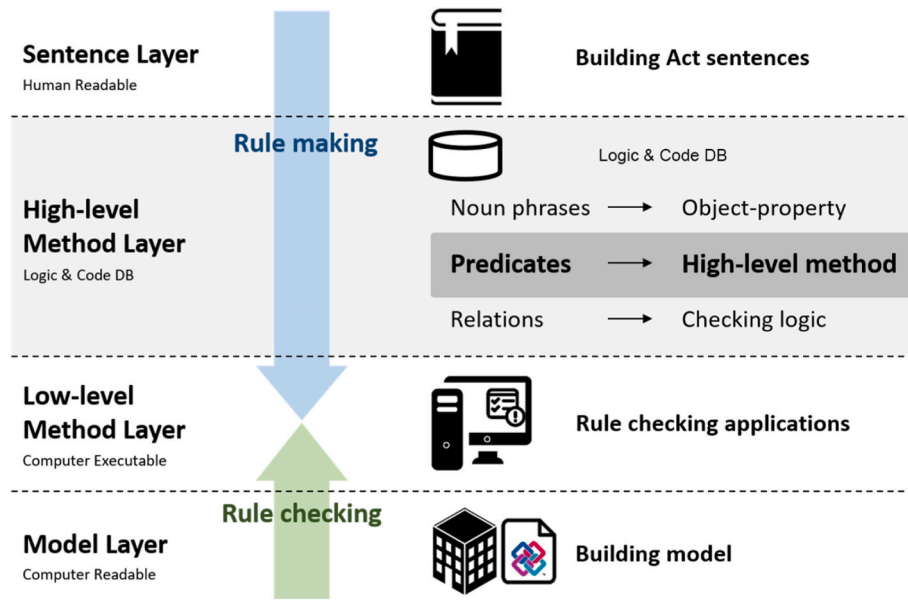


Fig. 1. Overview of the logic rule-based mechanism for the explicit representation; the dashed box in the middle indicates the focus of this study.

representations (Tenney et al., 2019). This flexibility contributes to BERT's ability to improve performance and adaptability to various tasks with minimal architectural modifications (Devlin et al., 2018). Additionally, the neural machine translation approach addresses the limitations of traditional fixed-length vector encodings, allowing for more accurate translations by dynamically searching for relevant parts of a source sentence (Bahdanau et al., 2015). These technical advances in NLP directly contribute to more efficient and versatile systems, addressing the shortcomings of labor-intensive implementations. Related methods include pre-trained models and efficient context making for unlabeled text, as well as creating and using domain-specific benchmarks, such as in successful cases in other specialized fields such as biomedicine and the legal field.

For example, in biomedicine, a pretrained model has been successfully implemented for translating PubMed and MIMIC-III clinical notes using BERT and ELMO models (Peng et al., 2019). In addition, research has shown that pretraining a language model from scratch is more effective than continuously pretraining a general-domain language model (Gu et al., 2021) in the field of biomedicine with abundant unlabeled text. Furthermore, attempts are being made to actively apply domain-specific NLP methods by creating a benchmark called BLURB (Biomedical Language Understanding & Reasoning Benchmark). Research in the legal field has been ongoing for a long time, and the latest techniques are being proposed to be domain-specific (Zhong et al., 2020). Furthermore, in this field, owing to its nature, there have been attempts to apply NLP methods for a long time, starting with studies that processed handcrafted rules and features from the days of severe computational limitations (Gardner, 1987). Research is underway to improve datasets and create benchmarks using deep-learning techniques (Chalkidis et al., 2019a, 2019b), and studies are ongoing to answer appropriate legal questions and summarize legal texts (Bhattacharya et al., 2019). Similarly to these cases from other fields, if the AEC-FM field can establish a standard language model that fits the domain jargon and logic, and efficiently build pre-trained models for unlabeled BIM objects and schemas using NLP technology, the latest NLP methods be directly applied, and the AEC-FM field can also address the shortcomings of previous approaches to ACCC, performing code checking processes in a less labor-intensive manner.

From this perspective, research in the AEC-FM field focuses on the entire ACCC process using NLP-based approaches in two ways: (1) interpreting the building code itself and (2) applying NLP to check the

compliance of the models. In the case of interpreting the building code itself, Beach et al. (2015) presented a rule-based semantic approach aiding domain experts in creating their own code compliance systems without requiring too much effort to create specific software. A review study by Yan et al. (2022) analyzed the application of text mining to a building code to translate it to computer-executable texts. Moon et al. (2022) tried to increase the precision of interpretation of building code by analyzing the different semantic properties with NLP. In addition, Zhou et al. (Zhou and El-Gohary, 2017) proposed an algorithm for automated energy requirement extraction from conservation codes using a combination of text classification methods, domain-specific preprocessing techniques, sequential dependency-based extraction methods, etc. Ventura et al. (2016) presented a parametric rule set for the Italian Construction Health and Safety norms using a BIM-based design and validation process.

Regarding the application of NLP to check the compliance of the models, Zhang et al. conducted research on extracting information from regulations and translating it to a computer-executable format (Zhang and El-Gohary, 2013). Two years later, they proposed a method to extract the information from the BIM model, and assessed it and compared it with the building code to check the compliance (Zhang and El-Gohary, 2015b). In the same year, they also researched transforming regulations to logic clauses directly used for automated reasoning (Zhang and El-Gohary, 2015a), and introduced a semi-automated approach for extending the Industry Foundation Classes schema with construction code information using semantic NLP and machine learning techniques (Zhang and El-Gohary, 2016) in the following year. Both studies highlight the potential of NLP to automate the translation of natural language into computable formats for ACCC in AEC-FM. In addition, Wu et al. (2023) attempted to make a new framework that reduces human labor and pursued a fully automated code checking system (I-SNACC).

However, utilizing NLP to automate the entire process of the Code Compliance Checking System (ACCC) still faces challenges (Guo et al., 2021). One significant issue is the ambiguity of interpretation arising from the complexity of describing space in language within building codes. As architecture is heavily influenced by local, cultural, and social contexts, the building codes that regulate it often exhibit national, regional, and cultural variations in the wording and application of similar elements. For example, in South Korea's building code (Korea Building Act, 2012), expressions such as "considering the surrounding

Name of the articles		Level of Possibility		
Number	(Building Act)	Possibility		
	[Enforcement Date 19. Oct, 2017.] [Act No.14792, 18	High	Mid	Low
1	Article 1 (Purpose)			○
2	Article 2 (Definition)			○
3	Article 3 (Exemption from Application)			○
4	Article 4 (Building Committees)			○
5	Article 5 (Relaxed Application)			○
6	Article 6 (Special Exceptions to Existing Buildings)		○	
7	Article 7 (Municipal Ordinances of Dos to Maintain ..			○

Fig. 2. Building-permit-related articles contained in the Korean Building Act and their potentiality for BIM-enabled checking in automation.

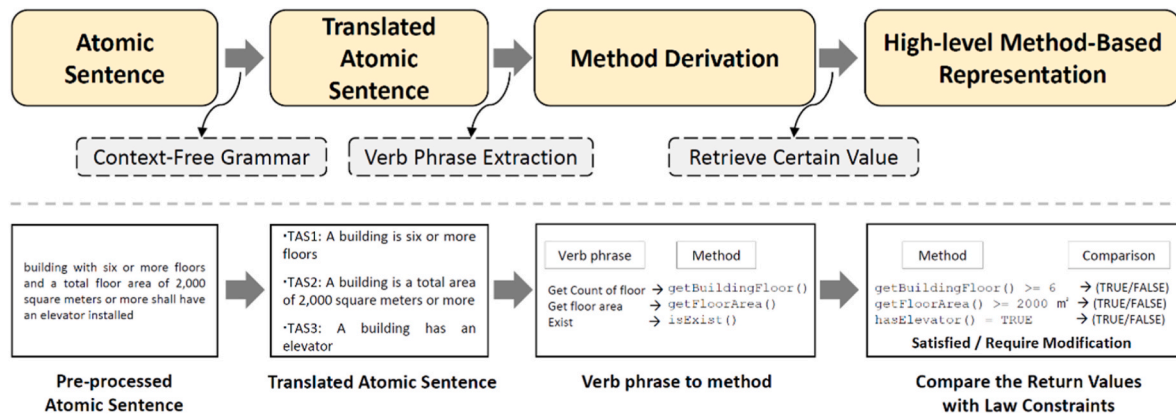


Fig. 3. Example of the proposed method-making approach for a given article.

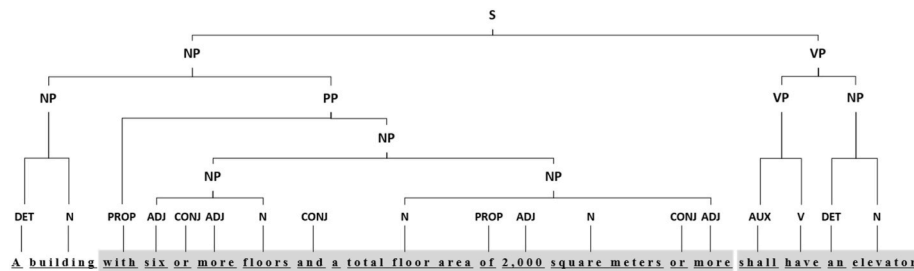


Fig. 4. Example of Korean Building Act sentence and its analysis with a parse tree (S: sentence; NP: noun phrase; VP: verb phrase; PP: prepositional phrase; DET: determiner; N: noun; PROP: proposition; ADJ: adjective; CONJ: conjunction; AUX: auxiliary verb; V: verb).

environment” necessitate human intervention. Moreover, because some building codes are arbitrarily cross-referenced, it is not possible to precisely translate the building code with the current status of NLP, without comprehending the relationships between clauses (Guo et al., 2021). With regard to this problem, Amor et al. (Amor and Dimyadi, 2021) argued that there are some unsuitable features in building acts

written in natural language that cannot be directly converted to computer processing; thus, it becomes the main obstacle for automated code compliance checking.

This problem degrades the credibility of computer-executable building codes, and makes them unable to establish general guidelines. Review studies (Ismail et al., 2017; Fuchs and Amor, 2021) have

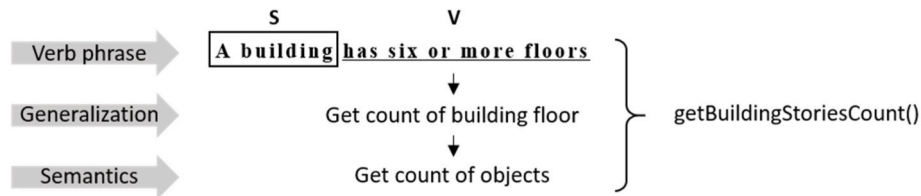


Fig. 5. An example of high-level method derivation from a given verb phrase.



Fig. 6. Example of different methods derived from an identical verb phrase from the contextual meaning: hasElevator(Building) and isExist(Elevator).

pointed out that current ACCC systems still have low scalability and limitations in automating the entire ACCC process. A lack of general guidelines leads to the absence of a unified conclusion in the ACCC domain (Aydın, 2022). This may adversely affect the interoperability between BIM and ACCC. Guo et al. (2021) highlighted the problem of hard-coding legislative information in a BIM model. A lack of interoperability leads to the ACCC system only being applied to partial processes and subtask construction. Fuchs et al. (Fuchs and Amor, 2021) noted the lack of studies on the whole process ACCC, and many studies are still conducted at sub-task levels, such as in construction safety field (Kincelova et al., 2019; Shen et al., 2022; Kim et al., 2012), at a national level (Zhou et al., 2022), and the level of fire safety of wooden structures (Kincelova et al., 2019). Therefore, there is a demand to develop higher-level NLP technology for the precise interpretation of building codes, thereby guaranteeing the integrity and credibility of the checking system.

3. High-level methods for representing explicit verb phrases

A logic rule-based mechanism consists of three parts: noun phrases (objects and properties), verb phrases (high-level methods for checking target building objects and properties), and logic (code generation processes). The conditional logic expressed in natural language sentences is translated via the logical process of clauses. Through this mechanism, a building act can be translated into an intermediate code that is its explicit representation. The intermediate code is manually generated and managed through a graphical user interface (GUI)-based application, which is stored in a database as a reusable source executable (Korean Building Act, 2022a, 2022b). Further details regarding the logic rule-based mechanism can be found in previous works (Lee et al., 2015c, 2016).

This study focuses on verb phrases, which are one part of a logic rule-based mechanism, and on the development of high-level methods. Fig. 1 presents an overview of the entire logic rule-based mechanism of the rule-making process. Verb phrases are important for two reasons: 1) they describe the conditions, properties, or related actions of the noun phrase (Bale, 2007), enabling them to represent the key functional requirements of building act sentences; and 2) some of them have specific semantic meanings. Therefore, verb phrases can be classified and processed into a more generalized form. Building objects and their properties can vary according to countries and their laws; the logical composition of a sentence also has distinct characteristics depending on the language. As an alternative, the generalized form of verb phrases can be applied to building-permit-related laws as well as various other types of laws in different countries.

This study adopted a method-based approach to explicitly represent generalized verb phrases. Method forms are widely used to represent

verb phrases; for example, Prolog (a descriptive language based on FOL) uses a method form to specify facts or define rules about objects and their relationships (Clocksin and Mellish, 2003). Here, high-level methods are proposed for the explicit representation of verb phrases in building act sentences. The semantic meanings of the verb phrases and building objects serve as functions of the high-level methods and arguments of these functions, respectively.

4. Method definition and representation approach

We used the Korean Building Act (Korean Building Act, 2022b) as an example source for the method definition process, reviewed the entirety of these laws, and analyzed their potential for BIM-enabled assessment, as shown in Fig. 2. A total of 113 articles were selected from 36 laws as candidates for this study, each possessing quantitative and explicit requirements for building design quality checking. Humans can generally understand the meaning of an article consisting of more than one sentence. However, it remains challenging to enable computers to correctly interpret the relationship between a main clause and a subordinate clause in a similar manner. Therefore, the Korean Building Act must be analyzed in minimum units. Atomic sentences (AS) are declarative sentences that cannot be broken down into simpler sentences (Ramanauskaitė et al., 2022). In this study, 113 target articles were divided into 1239 ASs; some of them were irrelevant to the body of law or did not contain building-permit-related content. Therefore, a filtering process was required to exclude such sentences and determine the target sentences. First, the ASs were sorted into two groups: those related to building permits and those not related; 273 sentences were excluded because they were not considered associated with building permit content. Second, those containing non-BIM data (11), including information on the outside of the site (roads, natural environments, etc.), were also filtered out. Third, ASs requiring excessive levels of building object properties were excluded (125), because they would be inefficient for automated reviews (Kim et al., 2014). Finally, ASs with qualitative information were also excluded (7); these sentences included vague or ambiguous expressions, such as “should be placed at appropriate intervals” or “should ensure sufficient illumination,” and cannot be automated. Through this process, 823 ASs (68% of the initially selected set) were identified as the target sentences.

The method definition and representation process can be generalized into four steps: AS preprocessing, sentence decomposition into translated atomic sentences (TASs), method derivation, and representation. Fig. 3 illustrates this process using a simple example. Each AS is pre-processed and parsed into a TAS using the CFG approach. Verb phrases are extracted from these TASs, translated into methods, and represented.

The following describes how high-level methods are derived from building codes in detail. Consider the following example AS from the

Table 1

High-level methods derived from the target building codes.

Representative Method	Description	Return Value	Extended Method Derived
getObject(obj)	query a certain object	Collection	getStair(), getRail(), getWindow(), getSpace(), ...
isExist(obj)	check the existence of a certain object	Boolean	
getObjectCount(obj)	get count of a certain object	Numeric	getSpaceCount(), getBuildingStoriesCount()
getObjectProperty(obj)	get basic property of an object	Collection	getObjectMaterial(), getDoorType(), ...
getObjectUse(obj)	get predefined use of an object	Collection	getBuildingUse(), getSiteUse()
getObjectType(obj)	get type of an object	Collection	getDoorType(), getWindowType(), ...
getObjectMaterial(obj)	get material information of an object	Collection	getWallMaterial(), getCoreStructureMaterial(), ...
getObjectHeight(obj, type)	get height of an object type: inside dimensions, centerline, outside dimensions	Numeric	getDoorHeight(), getBuildingHeight(), ...
getObjectWidth(obj, type)	get width of an object type: inside dimensions, centerline, outside dimensions	Numeric	getHallwayWidth(), getStairWidth(), ...
getObjectArea(obj, type)	get area of an object type: inside dimensions, centerline, outside dimensions	Numeric	getSpaceArea(), getFloorArea(), ...
getTotalFloorArea(obj, type)	get total area of a building type: inside dimensions, centerline, outside dimensions	Numeric	getWindowArea(), ...
getFloorAreaRatio(obj, type)	get floor area ratio of a building type: inside dimensions, centerline, outside dimensions	Numeric	
getBuildngToLandArea(obj, type)	get building-to-land area ratio of a building type: inside dimensions, centerline, outside dimensions	Numeric	
getObjectGradient(obj)	get gradient of an object	Numeric	getRampGradient(), ...
getObjectMaterialType(obj)	get predefined material type of an object	Collection	getWallMaterialType(), ...
isGrouped(obj, type)	check whether an object is partitioned with following type or not type: firewall, partition wall, fire-resistant construction, fireproof construction	Boolean	
isFireResistantCompartment(obj)	check whether an object is a fire-resistant compartment or not	Boolean	
isFireResistantConstruction(obj)	check whether an object is a fire-resistant construction or not	Boolean	
isFireProofConstruction(obj)	check whether an object is a fireproof construction or not	Boolean	
hasObject(obj)	check inclusion relation of objects	Boolean	hasSpace(), hasElevator(), hasWindow(), ...
getObjectDistance(obj a, obj b, type)	get distance between objects type: door to door, most remote point	Numeric	getSpaceDistance(), getWindowDistance(), ...
getObjectVerticalDistance(obj a, obj b, type)	get vertical distance between physical elements type: inside dimension, centerline, outside dimension	Numeric	
isConnectedTo(obj a, obj b)	check whether objects a and object b are physically connected or not	Boolean	
isExternal(obj)	check whether object a is	Boolean	
isAdjacent(obj a, obj b)	check whether object a and object b are not directly accessible but adjacent	Boolean	
isAccessible(obj a, obj b)	check whether object a and object b are accessible	Boolean	
isGoThrough(obj a, obj b, obj c)	check whether object a and object c are accessible through object b	Boolean	
getDirection(obj)	check direction of objects	Collection	getDoorSwingDirection()
isEgressDirection(obj)	check egress direction	Boolean	

Korean Building Act (Article 64, Clause 1), which regulates the installation of elevators:

“A project owner of a building with six or more floors and a total floor area of 2000 square meters or more shall have an elevator installed therein.”

● Step 1: Preprocessing

Noun phrases in ASs vary between regular building objects and non-AEC ones. Non-AEC building objects include actors, organizations, law names, etc. Building objects include tangible building elements (e.g., walls, slabs, and roofs), spatial objects, and dynamically instantiable objects (e.g., paths). Although both pertain to building permits, only building objects are relevant to method definition; therefore, sentences

with non-AEC building objects require modification. The preprocessing step appropriately modifies these objects and reflects the changes in the verb phrases. In this context, the example sentence could be modified as follows:

“A building with six or more floors and a total floor area of 2000 square meters or more shall have an elevator”

Compared with the original sentence, the subject is converted from an actor (a project owner) into a building object (a building). The verb phrase is also modified accordingly but still has the same meaning.

● Step 2: Translated Atomic Sentence (TAS)

A TAS provides a simplified sentence structure that enables clear extraction of verb phrases. The CFG concept was employed to analyze

Table 2
Hierarchical structure of the method classification.

Target	Property Type	Checking Type	Method	
			Representative	Extended
Obj Object Instance(s)		OQ Object Query	getObject()	getStair(), getRail(), getWindow(), getSpace(), ...
		OE Object Existence	isExist()	
Prop Property of Object Instance(s)	BP Basic Property	OC Object Count	getObjectCount()	getSpaceCount(), getElevatorCount(), getDirectStairCount(), ...
		PQ Property Query	getProperty()	getElevLiveLoad(), ...
		OT Object Type	getObjectType()	getDoorType(), getWindowType(), getStairType(), ...
		OM Object Material	getObjectMaterial()	getWallMaterial(), getCoreStructureMaterial(), ...
		OU Object Use	getObjectUse()	getsiteUse(), getBuildingUse(), getFloorUse(), getSpaceUse()
	GP Geometric Property	OH Object Height	getObjectHeight()	getDoorHeight(), getWindowHeight(), getSpaceHeight(), getStairStepHeight(), getBuildingHeight(), ...
		OW Object Width	getObjectWidth()	getHallwayWidth(), getStairWidth(), ...
		OA Object Area	getObjectArea()	getSpaceArea(), getFloorArea(), getBuildingArea(), getWindowArea(), ...
			getFloorAreaRatio() getBuildingToLandArea() getObjectGradient()	getRampGradient()
	DP Derived Property	OG Object Gradient		
		MT Material Type	getObjectMaterialType()	getWallMaterialType(), getCoreStructureMaterialType(), ...
		FS Fire Safety	isFireResistantCompartment() isFireResistantConstruction() isFireProofConstruction()	
	RP Relational Property	IC Inclusion	hasObject()	hasSpace(), hasElevator(), hasDirectStair(), ...
		DT Distance	getDistance() getVerticalDistance() isConnectedTo() isGrouped() isExternal() isAdjacent() isAccessible() isGoThrough()	getSpaceDistance(), getWindowDistance(), ...
		CT Connectivity		
		PT Path		
		DT Direction	getObjectDirection() isEgressDirection()	getDoorSwingDirection()

ASs and extract TASs. CFG is a widely accepted framework that provides an effective means of analyzing natural language and defining well-formed expressions for computation (Lee et al., 2006). Preprocessed sentences are parsed and transformed into a subject–verb (SV) or a subject–verb–object (SVO) structure. The modification is recursively performed until the atomic SV or SVO structure is rendered. Fig. 4 illustrates the grammatical analysis of an example sentence as an intermediate step in the TAS extraction process.

According to this analysis, the example sentence contains a prepositional phrase (PP) and a verb phrase (VP). The former modifies the subject and the latter explains its associated action.

PP: A building with six or more floors and a total area of 2000 square meters or more (1)

VP: A building has an elevator (2)

This PP can be further segmented into smaller atomic SVO structures. Thus, the TASs of the example sentence are as follows:

TAS 1: A building has six or more floors (3)

TAS 2: A building has a total area of 2000 square meters or more (4)

TAS 3: A building has an elevator (5)

Each TAS has a simple SV or SVO structure with a clear verb phrase denoting (from the top down) the number of floors, total area, or existence of a specific element in a building. In this study, 1588 TASs were derived from the 823 ASs of the Korean Building Act.

● Step 3: Method Derivation

Deriving methods from verb phrases is the process of representing their semantic meaning; the verb phrases are interpreted, generalized, and transformed into high-level methods. Fig. 5 illustrates this process for TAS 1.

The following methods can be derived from each TAS of the example

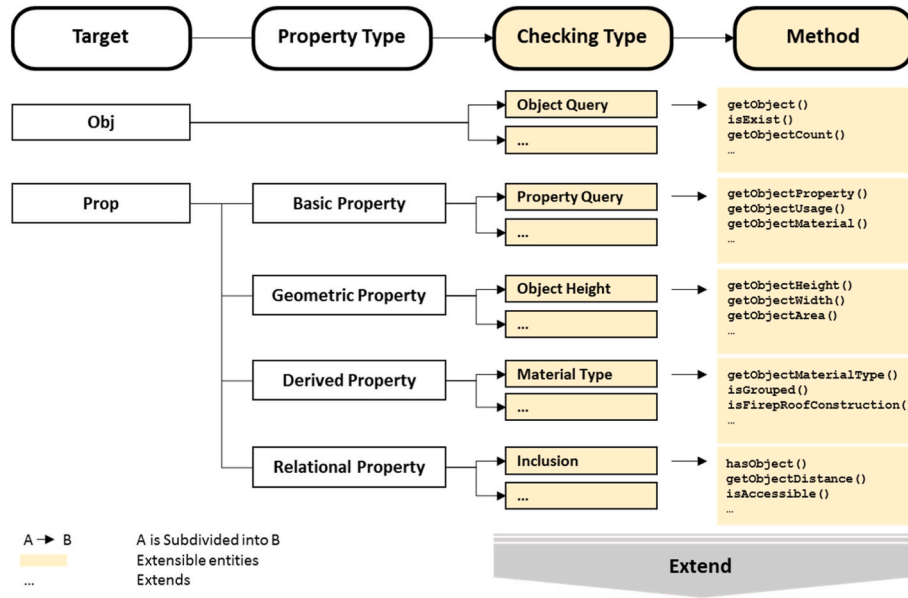


Fig. 7. Extensibility of the high-level implementable methods proposed in this paper.

sentence:

`getBuildingStoriesCount()` (6)

`getTotalFloorArea()` (7)

`hasElevator()` (8)

The method name complies with a consistent convention. Its style follows that of Java, one of the most widely utilized programming languages (TIOBE, 2019). It consists of multiple words and should begin with a verb (in lowercase) followed by a noun, adjective, etc. The first letters of the second and following words should be capitalized (Oracle and The Java Tutorials, 2022). The method name is written on the left, and its parameters are in parentheses on the right. Extended Backus-Naur form (EBNF) notation (Extended BNF, 1497) is used for method definition as follows:

Method M = (verb, obj, [prop], [cond] | verb, prop | verb, cond), “(”, [params], “)”;

where verb is the infinitive form of a verb, obj is an object instance in a building, prop is a subordinate property of a building object instance, cond is a condition of an object instance or property, and params is a parameter that consists of an object instance and the type of checking.

The method name begins with an infinitive verb form related to the return value type. There are three types of verb form: get, is, and has. The methods beginning with “get” return a collection of objects, strings, and numerical data, while those beginning with “is” or “has” return values of either “true” or “false.”

The parameter is also an important component of the method because it specifies which object will be checked (obj) and how this will be done (type). It is defined as follows:

Params = obj, [obj], [type]; (10)

`getFloorArea(“LivingRoom”, b)` is a good example of a parameter. The target of this method is all the living rooms in a given building. The letter “b” indicates the type of criterion used for calculating the area. The floor area varies depending on whether it is measured by the inside dimensions (type a), centerline (type b), or outside dimensions (type c) of the wall object. Because area calculations are a major factor in building permit laws, the type of compliance checking should be categorized accordingly.

In some cases, an identical verb phrase can be represented in different ways because interpretation may vary with the context. For example, the verb phrase of TAS 3, “has,” can be interpreted in two ways, as shown in Fig. 6; the derived methods represent each of the single meanings.

● Step 4: High-Level Method-Based Representation

High-level methods retrieve certain values from a given building model. The rules are checked to compare the return value of a method with the constraints stated in a legal sentence. For example, the sentence “A building shall have six or more floors” states the constraint “six or more floors.” If the method `getBuildingStoriesCount()` retrieves the number of floors from a given building model and its value is greater than or equal to six, this rule is determined to be true; the opposite results in the false condition. Comparisons between the return values and constraints are the defining atomic units of rule checking. Because appropriate logic is required to represent these comparisons, the ALU concept was employed to represent this logic. An ALU is a digital electronic circuit that performs arithmetic and bitwise logical operations (Kirichenko et al., 2019). In this study, this represents a declarative clause that acts as an atomic unit for rule checking. The ALU consists of three parts: the left operand, operator, and right operand (Park et al., 2015). High-level methods are placed in the left operand and compared with explicit values in the right operand using comparison operators. The ALUs for the example TAS are as follows:

`getBuildingStoriesCount() >= 6` (12)

`getTotalFloorArea() >= 2000m2` (13)

`hasElevator() = TRUE` (14)

Each representation can be determined as true or false. If it is true, the given building model satisfies the rule; otherwise, modifications are required.

5. Method classification and extensibility

5.1. Method classification

There are two types of high-level methods: representative and extended methods. Representative methods are the basic forms and can

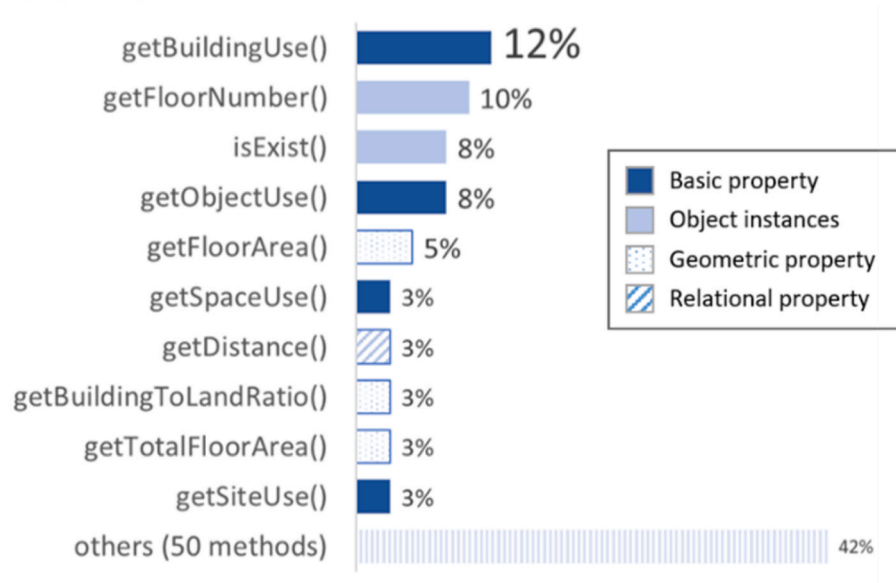
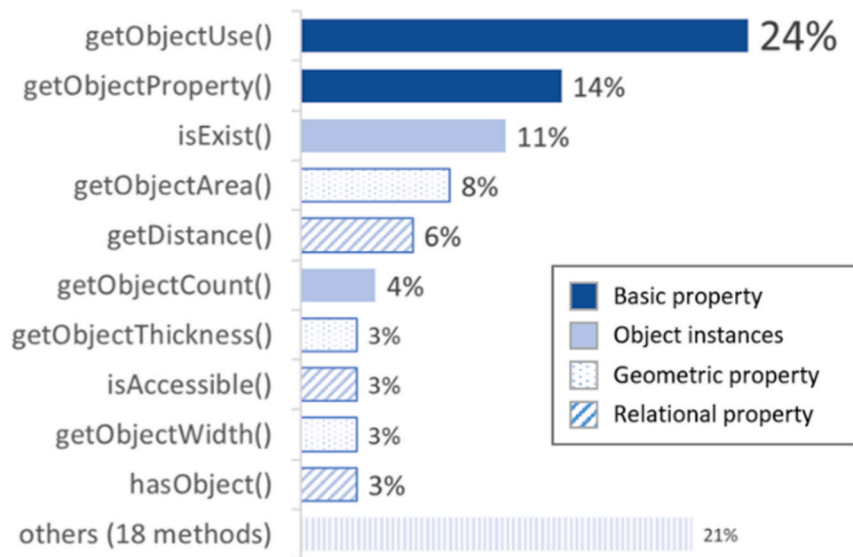
(A) Frequency of all 60 methods**(B) Frequency of the 28 representative methods**

Fig. 8. The 10 most frequently used methods and their usage percentages among all 60 methods (above) and the 28 representative ones (below) found in the 823 sentences considered in this study.

be extended according to the specific type of object and properties checked. In the considered example, 32 extended methods were derived from 28 representative methods; hence, a total of 60 different methods were derived from the target sentences. Table 1 summarizes the derived methods and describes their parameters and return values.

This method classification has a four-level hierarchy: target, property type, checking type, and method. It was established using a bottom-up approach. Furthermore, the derived methods were grouped to form a supertype. An extended method is classified as being representative because the relationship between the two is a subdivision of the specific checking object and property. Methods directly related to the verb phrase and checking type are associated with groups of verb phrases, as shown in Table 2. In this example, there were 16 types of method-processable groups of phrases. Once the specifying property was clarified, 19 checking types were established, each representing a subtype of

the representative methods. Checking types are divided into two groups—object instances and objects with their properties—according to the target of the checking content. The first group returns results including only object instances; methods that query objects, check their existence, and/or count the number of objects that are part of this group. The second group examines various properties of the object instances and returns the results. According to the property type, these methods are further divided into four subtypes: basic, geometric, complex, and relational.

A basic property is the default property inherent in an object (except for geometric information), such as its name, window type, etc. Geometric properties are related to the geometric information of an object, including the height, width, and area. A complex property is regulation-specific, meaning that its specifications are defined in the regulation. Complex properties can be completely checked in two ways. The first is

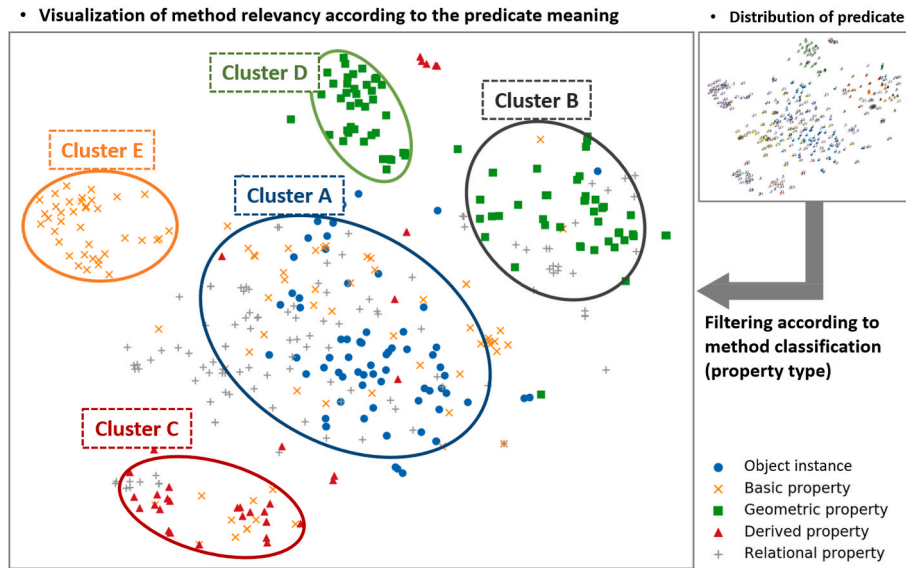


Fig. 9. t-Stochastic neighbor embedding visualization of the predicate distributions in a 200-dimensional vector space.

Table 3

Translation of Korean Building Act, Article 64, Clause 1 (pertaining to elevator installation in buildings) into KBimCode. This clause was exemplified in Section 4.

Law Sentence	[Korean Building Act, Article 64, Clause 1] "A project owner of a building with six or more floors and a total floor area of 2000 square meters or more shall have an elevator installed therein."	
Sentence Processing	AS	NP 1. Building with six or more floors and a total floor area of 2000 square meters or more VP 2. Have an elevator
	TAS	TAS 1. Building is six or more floors TAS 2. Building is a total floor area of 2000 square meters or more TAS 3. Building has an elevator
High-Level Methods	Method 1. <text-color color="#FF0000;">getBuildingStoreisCount() Method 2. <text-color color="#FF0000;">getTotalFloorArea() Method 3. <text-color color="#FF0000;">isExist(elevator)	
KBimCode	<pre> <text-color color="#FF0000;">IF (<text-color color="#FF0000;">getBuildingStoriesCount()<text-color color="#FF0000;"> >=6 <text-color color="#FF0000;">AND <text-color color="#FF0000;">getTotalFloorArea()<text-color color="#FF0000;"> >=2000m2) <text-color color="#FF0000;">THEN<text-color color="#FF0000;"> isExist(<text-color color="#FF0000;">elevator<text-color color="#FF0000;">)<text-color color="#FF0000;"> = TRUE <text-color color="#FF0000;">END IF </pre>	

the realization of a BIM guide that contains the values of every object embedded into the building models; this type of checking is simple but requires considerable effort to make building models that satisfy every prerequisite in the modeling guide. The second method uses programming and logic to derive the requisite data at the implementation level. Although the implementation complexity increases, this method might reduce the burden on users and avoid errors and inconsistencies in the model. Therefore, finding a trade-off between these two options is another challenge that must be addressed when executing high-level methods in this category. Finally, relational properties address the relationships between multiple objects. Within the scope of this study, five types of relationships were specified: inclusion, distance, connectivity, path, and direction. As the target sentences are extended to other laws, new types of relations and subdivided methods will be developed.

5.2. Extensibility

The high-level methods proposed in this paper were developed from a portion of the building permit requirements in the Building Act. As the range of targeted codes is broadened, the methods will be extended in two ways, as shown in Fig. 7. The first is the extension of the existing methods in terms of their names and parameters, which will increase the number of methods in use but may not affect their method classification

structure. The second is the extension of the semantic meanings represented by new methods, which will affect the extension of the checking type; in addition, the new methods may form new categories for themselves.

As shown in Table 2, the majority of high-level methods are related to the properties of various objects. Within the range of high-level methods, new methods to check any object property can be developed without limit; however, since this can increase the number of methods very quickly, alternative expressions are necessary to express rare properties. For example, the property "accommodating building capacity" appeared only once among the 823 sentences considered in this study. This property is essential for checking the related requirements; however, in this case, it can be expressed alternatively via dot notation as `Building.occupancy ≥ 60`.

6. Evaluation and demonstration

6.1. Frequency and relevancy of the methods

This section describes the frequency and relevancy analyses of the methods derived from the 823 sentences, which are used as supportive data for method pre-validation. Fig. 8 shows the frequency of all 60 methods and the 28 representative ones found in this study. The

Table 4

Translation of the Enforcement Decree of the Korean Building Act, Article 34, Clause 1 (pertaining to the regulation of direct stair installation) into KBimCode.

Law Sentence		[Enforcement Decree of the Korean Building Act, Article 34, Clause 1] “On each floor of a building, direct stairs leading to the shelter floor or the ground other than the shelter floor shall be installed such that the walking distance from each part of the living room to the stairs is not more than 30 meters...”
Sentence Processing	AS	AP 1. Other than the shelter floor NP 2. Direct stairs leading to the shelter floor or the ground VP 3. Direct stairs are installed such that the walking distance from each part of the living room to the stairs is not more than 30 meters
	TAS	TAS 1. The floor is not the shelter floor TAS 2-1. The direct stairs are accessible via the shelter floor TAS 2-2. The direct stairs are accessible via the ground TAS 3-1. The floor has a living room TAS 3-2. The walking distance from the most remote point of the living room to the direct stairs is not more than 30 meters
High-Level Methods		Method 1. getObjectProperty() Method 2. isAccessible() Method 3-1. hasSpace() Method 3-2. getDistance()
KBimCode		<pre> IF (getObjectProperty(Floor) != "ShelterFloor") THEN Floor myFloor { getObjectProperty(Floor) != "ShelterFloor" } Space myLivingRoom { hasSpace(myFloor, LivingRoom) = TRUE } Stair myStair { isAccessible(Stair, ShelterFloor) = TRUE OR isAccessible(Stair, Ground) = TRUE } getDistance(myLivingRoom, myStair, b) <= 30m //b stands for distance measurement by the most remote point ENDF </pre>

‘getBuildingUse’ method (12%) was the most frequent among all 60 methods, followed by ‘getFloorNumber’ (10%), ‘getObjectUse’ (8%), ‘isExist’ (8%), and ‘getFloorArea’ (5%). The remaining 55 methods covered 57% of the target sentences with an even distribution. All the methods play critical roles in the checking of target sentences. The frequency of the representative methods were each estimated as the sum of the subordinate extended methods. The most frequent was ‘getObjectUse’ (24%), followed by ‘getObjectProperty’ (14%), ‘isExist()’ (11%), ‘getObjectArea’ (8%), and ‘getDistance’ (6%). These results show that the methods checking building object properties had a higher usage frequency; among them, those checking the basic properties (such as ‘getObjectUse’ and ‘getObjectProperty’) accounted for more than 38% of the entire frequency. Implementation of high-level methods in rule checking software incurs many challenges. Various libraries are related to the implementation of one high-level method; for example, geometric methods, IFC object and property querying, and object mapping. The frequency analysis provides useful data to identify which method should be initially implemented when developing such software.

The semantic relevancies of the verb phrases extracted from the 823 sentences were also analyzed via neural net-based word embedding techniques (Mikolov et al., 2013). Word embedding is a language model representing words as vector values based on their semantic meanings. In this study, each verb phrase was translated into a 200-dimensional vector of real numbers using a pre-trained word embedding model implemented with a Python package library (Rehurek and Sojka, 2010). Fig. 9 displays a t-stochastic neighbor embedding visualization for the distribution of these verb phrases in the 200-dimensional vector space; in this visualization, each verb phrase is annotated with the

representative method corresponding to its semantic meaning. Five clusters of relevant methods can be identified. Cluster A is the largest cluster and includes the most frequently used methods of ‘isExist’ and ‘getObjectUse’; it also clearly contains ‘isAccessible’ and ‘isConnectedTo’, which both check the relational properties of the objects. Cluster B includes methods such as ‘getObjectHeight’ and ‘getObjectDistance’, which calculate the geometric shape of an object or the distance between target objects. Cluster C consists of methods associated with fire safety, such as ‘isFireResistantStructure’, ‘isFireResistantCompartment’, and ‘getObjectMaterial’. Cluster D is mainly composed of ‘getObjectArea’; it also includes ‘isFireResistantCompartment’ because some regulations require the installation of a fire-resistant compartment within a certain range of floor area.

The relevancy analysis helps to determine the priority of method implementations and their sequences of development. As demonstrated in each cluster, relevancy between methods could be interpreted as the possibility for checking associated properties or appearing simultaneously at a single checking rule. Therefore, its determination could allow the re-categorization of the methods in terms of implementation. For example, the methods checking basic properties are regarded as being top priority for implementation in string-based frequency analysis. In contrast, those checking relational properties are considered as being low priority. However, the results of the relevancy analysis suggest that they have a semantic relationship and can therefore be implemented simultaneously, regardless of frequency. Likewise, in other cases, the relevancy analysis helps developers to implement related methods that are required together to check specific criteria.

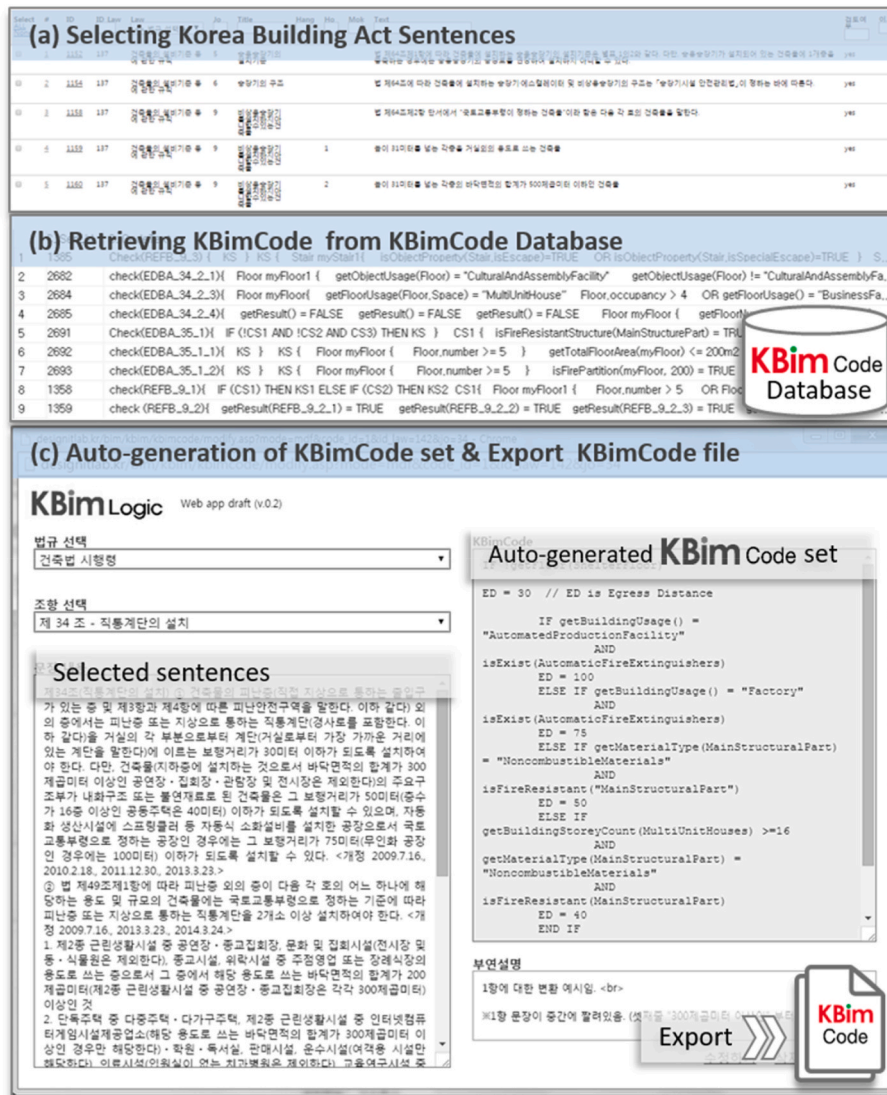


Fig. 10. Screenshot of the KBimLogic application; by selecting target articles and sentences, KBimCode can be automatically generated using its database.

6.2. Building permit requirements represented by the code database

This section demonstrates some translated rule examples using the simple principle established earlier: rules are translated from Korean Building Act sentences into intermediate code. While generating KBimCode, we used high-level methods and other components of the logic rule-based mechanism, such as the object-property for representing noun phrases and the logic for representing the sentence relationship. Although these two components are outside the scope of this study, they relate to high-level methods and are essential for completing KBimCode; for example, object-property is used as a parameter of high-level methods that supports a reliable review of Korean Building Act-specific objects and their properties in a BIM environment. Related issues have previously been discussed in detail (Lee et al., 2015c).

Tables 3 and 4 show a small part of the translation process from Korean Building Act sentences into KBimCode. Table 3 summarizes the translation of Article 64, Clause 1 (used as the example sentence in Section 4); this clause was already translated to the ALU-level in Section 4, but Table 3 shows the complete pseudo-code containing a logical composition. As another example, Table 4 shows a demonstration of the Enforcement Decree of the Korean Building Act, Article 34, Clause 1, which regulates the emergency egress distance of direct stairs.

Although only some code examples are reported in this paper, all 823

target sentences were translated into KBimCode by domain experts and used to develop a database (Lee, 2023a). Users can choose articles within the building permit-related content of the Korean Building Act, as shown in the 'Law sentence' of Tables 3 and 4, rather than typing or composing code directly. The use of KBimCode and its database enables specific purpose rules to be acquired, even without knowing the high-level methods described in this paper; users can easily obtain the appropriate rules for their specific purpose in the editing application. As overviewed in Fig. 10, KBimLogic provides an interface for selecting target articles and specific sentences. Then, the selected sentences are logically combined with pre-defined sentence relationships; in other words, sentences with complex cross-references can be composed automatically into a series of KBimCode commands. The KBimCode series can be exported by KBimLogic as a computer-executable rule set file. The actual output file was developed using the software-independent formats of JSON (JSON, 2023) and XML (XML and W3C, 1114).

6.3. Demonstration of automated compliance checking

This section briefly describes an example of automated code compliance checking using KBimCode (Lee, 2023a), and an actual test model based on a demonstration using rule-checking software. The

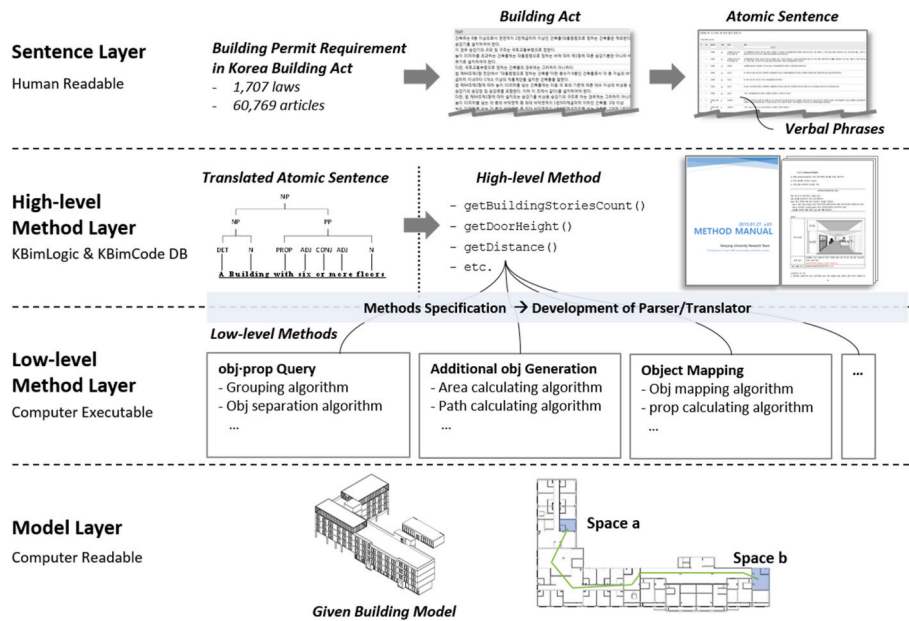


Fig. 11. Schematic of the four interlinked layers in the high-level method development process: the top two are considered in this study.

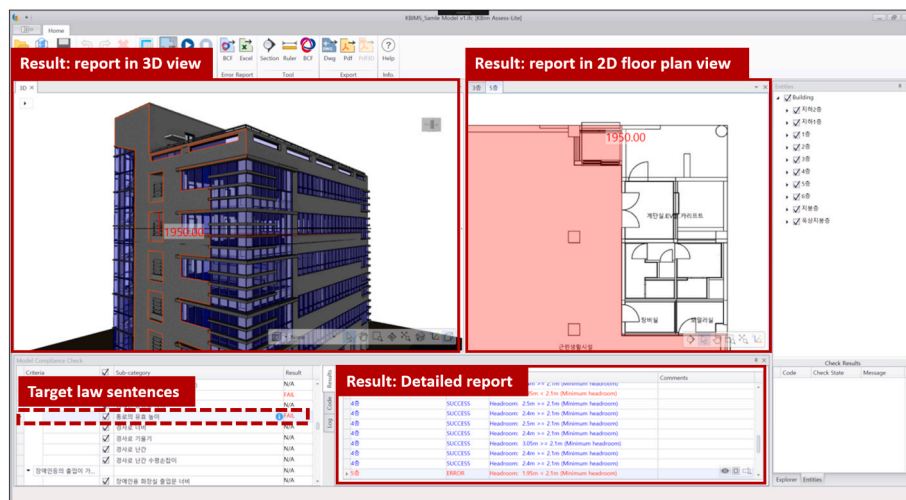


Fig. 12. Example of code compliance checking in KBimAssess with KBimCode; the dashed box in the bottom left indicates the target legal sentences (Cospec Innolab, 2023).

feasibility of KBimCode was verified using actual rule-checking software called KBimAssess, which handles IFC-based models and is specialized for the verification of building-permit-related requirements of the Korean Building Act. As shown in Fig. 11, there are four layers in the development of high-level methods. The top two layers are the most pertinent to this study, whereas the other two mainly deal with implementation issues. The method specifications (as well as the KBimCode syntax) are documented in a manual and provided to software vendors (Specification of the high; Lee, 2023b); they include the method name, description, return value type, syntax, parameter type, and pre-conditions. The parsing library and translator for performing the mapping between high- and low-level method sets were developed based on these specifications. In the case of KBimAssess, KBimCode is translated into executable low-level program code using its own parsing library. For instance, the high-level method “getObjectDistance” is associated with more than 10 types of low-level geometric methods in KBimAssess.

Fig. 12 shows an example of KBimCode execution in KBimAssess.

KBimCode was exported into the JSON file format (JSON, 2023) using the KBimLogic application and executed in KBimAssess. For this demonstration, an IFC model of an actual office building in Seoul, South Korea was used as the test model; this building has a total floor area of 5734 m² with four basement floors and 15 aboveground floors. The checking results return a pass or fail condition with a visual report and a detailed text report.

This test validated the feasibility of KBimCode and high-level method execution using rule-checking software. High-level methods provide software vendors with the advantage of reduced difficulty in interpreting building acts. Moreover, they can provide supporting information for prioritizing the development of low-level libraries for automated assessments.

7. Conclusions

This paper presents an approach involving a method for defining high-level methods mainly focused on verb phrase translation, providing

useable techniques that allow software developers to enhance their low-level rule verification processes. The accurate representation of design requirements is crucial in the early stages of construction projects, where architects, code specialists, and developers must work together. To increase the generalizability, neutralization, and reusability of rules stated in natural language, we propose a series of high-level computer programming methods (operators). We classified building-permit-related regulations from the Korean Building Act using a filtering process and identified 823 as example targets. Verb phrases explaining the condition and associated action of an object were extracted from 16 types, which were processed via a method-based approach. The ASs were transformed into TASs to clarify the functions and arguments of the methods, which were also derived from the verb phrases specified in the TASs. In total, 28 representative methods and 32 extended methods were derived from the full set of target sentences. These methods were classified according to a four-level hierarchy: target, property type, checking type, and method. First, the method targets were divided into two types: object instance(s) or objects and their properties. Second, the object properties were categorized into four types: basic, geometric, complex, and relational properties. Third, the checking types specified by the building code formed their own criterion. Fourth, each criterion was subdivided into various methods. The third- and fourth-level methods could be extended with the advent of new types of verb phrases and building objects.

The proposed methods translate verb phrases into building code sentences, then retrieve data from a given building model. Their return values can be real numbers or true/false conditions. By employing the ALU concept, a comparison between return values and constraints is logically expressed in an intermediate form. Once restructured using IF-THEN-ELSE logic, a set of ALUs completes the intermediate code for rule translation. The intermediate code is processed into open-text code and can be utilized in various rule-checking applications; in addition, its accumulated database is reusable, allowing efficient rule checking. A demonstration of actual rule checking proved the feasibility of KBim-Code and of the high-level methods.

The method proposed in this paper is more efficient in terms of time, effort, and cost than building low-level systems, and by providing a universal standard for the different building codes of each country, it is possible to secure international consensus and develop a more efficient BIM-based Code Compliance Checking System. The high-level scheme presented in this paper, which simultaneously interprets verb phrases and building acts, provides additional syntax and semantics for BIM code checking, which will help resolve the aforementioned problems. Additionally, this high-level scheme overcomes the structural limitations, such as complex relations and integrity, of low-level BIM implementation models that cannot incorporate the latest natural language models into BIM code-checking tasks. By bridging this gap, the possibility of applying new NLP methodologies, such as large language models (LLMs), can be achieved, and the real-time application of regulations to be considered at each design phase in the BIM-based design process will result in efficient project duration reduction and construction cost savings.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work was supported by the National Research Foundation of

Korea (NRF) grant funded by the Korea government (MIST) (No. NRF-2021R1A4A1032306 and No. NRF-2022R1A2C1093310).

References

- Amor, R., Dimyadi, J., 2021. The promise of automated compliance checking. *Dev. built environ.* 5, 100039.
- Aydin, M., 2022. A review of BIM-based automated code compliance checking: a meta-analysis research. *Autom. Control. Theor. Appl.* 39.
- Bahdanau, D., Cho, K., Bengio, Y., 2015. Neural Machine Translation by Jointly Learning to Align and Translate arXiv preprint arXiv:1409.0473.
- Bale, A.C., 2007. Quantifiers and verb phrases: an exploration of propositional complexity. *Nat. Lang. Ling. Theor.* 25, 447–483. <https://doi.org/10.1007/s11049-007-9019-8>.
- Barwise, J., 1977. An introduction to first-order logic. In: *Handbook of Mathematical Logic (Studies in Logic and the Foundations of Mathematics)*. Jon Barwise.
- Beach, T.H., Rezgui, Y., Li, H., Kasim, T., 2015. A rule-based semantic approach for automated regulatory compliance in the construction sector. *Expert Syst. Appl.* 42 (12), 5219–5231.
- Bhattacharya, P., Hiware, K., Rajgaria, S., Pochhi, N., Ghosh, K., & Ghosh, S. (2019). A comparative study of summarization algorithms applied to legal case judgments. In *Advances in Information Retrieval: 41st European Conference on IR Research, ECIR 2019, Cologne, Germany, April 14–18, 2019, Proceedings, Part 1* 41 (pp. 413–428). Springer International Publishing.
- Chalkidis, I., Androutsopoulos, I., Aletras, N., 2019a. Neural Legal Judgment Prediction in English arXiv preprint arXiv:1906.02059.
- Chalkidis, I., Fergadiotis, M., Malakasiotis, P., Androutsopoulos, I., 2019b. Large-scale Multi-Label Text Classification on EU Legislation arXiv preprint arXiv:1906.02192.
- Choi, J., Kim, H., Kim, I., 2015. Open BIM-based quantity take-off system for schematic estimation of building frame in early design stage. *J. Comput. Des. Eng.* 2 (1), 16–25.
- Clocksin, W.F., Mellish, C.S., 2003. Programming in Prolog. Springer-Verlag.
- Cospec InnoLab. KBim assess - lite. <http://www.inno-lab.co.kr>. (Accessed 3 February 2023).
- Devlin, J., Chang, M.W., Lee, K., Toutanova, K., 2018. Bert: Pre-training of Deep Bidirectional Transformers for Language Understanding arXiv preprint arXiv:1810.04805.
- Ding, L., Drogemuller, R., Rosenman, M., Marchant, D., Gero, J., 2006. Automating code checking for building designs. In: Brown, K., Hampson, K., Brandon, P. (Eds.), *Clients Driving Construction Innovation: Moving Ideas into Practice*. CRC for Construction Innovation, pp. 113–126.
- E-Info, CORENET Singapore. <http://www.corenet.gov.sg/>. (Accessed 15 September 2022).
- Eastman, C.M., Teicholz, P., Sacks, R., Liston, K., 2008. *BIM Handbook: a Guide to Building Information Modeling for Owners, Managers, Designers, Engineers, and Contractors*. John Wiley & Sons.
- Eastman, C., Lee, J.-M., Jeong, Y.-S., Lee, J.-K., 2009. Automatic rule-based checking of building designs. *Autom. Construct.* 18 (8), 1011–1033.
- EDM: EXPRESS Data Manager. EPM technology. <http://www.epmtech.jotne.com>. (Accessed 23 September 2022).
- Extended BNF. ISO/IEC 14977 : 1996(E). <http://www.cl.cam.ac.uk/~mgk25/iso-14977.pdf>.
- Fuchs, S., Amor, R., 2021. Natural language processing for building code interpretation: a systematic literature review. In: *Proc. Of the Conference CIB W78*, pp. 11–15, 2021.
- Gardner, A.V.D.L., 1987. *An Artificial Intelligence Approach to Legal Reasoning*. MIT press.
- Getuli, V., Ventura, S.M., Capone, P., Ciribini, A.L., 2017. BIM-based code checking for construction health and safety. *Procedia Eng.* 196, 454–461.
- Greenwood, D., Lockley, S., Malsane, S., Matthews, J., 2010. Automated compliance checking using building information models. In: *Proceedings of the Construction, Building and Real Estate Research Conference of the Royal Institution of Chartered Surveyors*.
- Gruber, T.R., 1993. A translation approach to portable ontology specifications. *Knowl. Acquis.* 5 (2), 199–220.
- Gu, Y., Tinn, R., Cheng, H., Lucas, M., Usuyama, N., Liu, X., et al., 2021. Domain-specific language model pretraining for biomedical natural language processing. *ACM Trans. Comput. Healthcare (HEALTH)* 3 (1), 1–23.
- Guo, D., Onstein, E., La Rosa, A.D., 2021. A semantic approach for automated rule compliance checking in construction industry. *IEEE Access* 9, 129648–129660.
- Han, C., Kunz, C.J., Law, K.H., 1997. Making automated building code checking a reality. *Facilit. Manag. J.* 22–28.
- Hopcroft, J.E., Ullman, J.D., 1979. “Chapter 4: context-free grammars”, “chapter 6: properties of context-free languages”. In: *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley.
- Hutchins, J., 2005. *The History of Machine Translation in a Nutshell* retrieved December, 20, 2009), URL: <http://www.hutchinsweb.me.uk/Nutshell-2005.pdf>.
- Ismail, A.S., Ali, K.N., Iahad, N.A., 2017. A Review on BIM-Based Automated Code Compliance Checking System. *IEEE*, pp. 1–6. ICRIIS.
- ISO TC184/SC4, 1997. *Industrial Automation Systems and Integration—Product Data Representation and Exchange—ISO 10303-11: Description Methods: the EXPRESS Language Reference Manual*. ISO Central Secretariat.
- Json, E.C.M.A.-404. *The JSON data interchange standard*. <http://www.ecma-international.org/publications/files/ECMA-ST/ECMA-404.pdf>. (Accessed 18 January 2023).
- Kim, I., 2015. *Open BIM Based Technological Environment for Building Design Quality Enhancement*. Kyung Hee University (Project report).

- Kim, I., Kim, Y., Choi, J., 2014. Building code typology and application for open BIM based code checking. *Korean J. Comput. Des. Eng.* 19 (3), 22–235.
- Kim, Y.R., Lee, S.H., Park, S.H., 2012. Development of Rule-Set Definition for Architectural Design Code Checking based on BIM - for Act on the Promotion and Guarantee of Access for the Disabled, the Aged, and Pregnant Women to Facilities and Information -. *Korean Journal of Construction Engineering and Management* 13 (6), 143–152.
- Kincelova, K., Botton, C., Blanchet, P., Dagenais, C., 2019. BIM-Based Code Compliance Checking for Fire Safety in Timber Buildings: A Comparison of Existing Tools.
- Kirichenko, A.F., et al., 2019. ERSFQ 8-bit parallel arithmetic logic unit. *IEEE Trans. Appl. Supercond.* 29 (5), 1–7. <https://doi.org/10.1109/TASC.2019.2904484>.
- Koo, B., Shin, B., 2018. Applying novelty detection to identify model element to IFC class misclassification on architectural and infrastructure Building Information Models. *J. Comput. Des. Eng.* 2 (1), 391–400.
- Korea building act, Article 11 section 4(1). https://elaw.klri.re.kr/eng_service/lawView.do?hseq=31602&lang=ENG-. (Accessed 12 April 2023).
- Korean building act, national statute data center Korea. <http://law.go.kr/main.html>. (Accessed 5 October 2022).
- Korean building act (English), statutes of Republic of Korea. http://elaw.klri.re.kr/kor_service/main.do. (Accessed 6 October 2022).
- Lee, J., 2010. Automated Checking of Building Requirements on Circulation over a Range of Design Phase. Georgia Institute of Technology (Ph.D. dissertation).
- Lee, J.-K., 2012. Building Environment Rule and Analysis (BERA) Language and its Application for Evaluating Building Circulation and Spatial Program. Georgia Institute of Technology (Ph.D. dissertation).
- J. K. Lee, KBimCode Database (accessed 4 January 2023) dataset.
- Lee, J.K., Specification of KBimCode syntax, KBimCode syntax manual. <http://designitlab.kr/bim/KBimCodeSyntaxManual.asp>. (Accessed 10 January 2023).
- Lee, J.K., Eastman, C.M., 2019. Demonstration of BERA language-based approach to offsite construction design analysis. In: *Offsite Production and Manufacturing for Innovative Construction*. Routledge, pp. 129–162.
- Lee, G., Eastman, C.M., Sacks, R., Navathe, S.B., 2006. Grammatical rules for specifying information for automated product data modeling. *Adv. Eng. Inf.* 20, 155–170.
- Lee, J.-K., Lee, J., Jeong, Y.-S., Sheward, H., Sanguinetti, P., Abdelmohsen, S., et al., 2012. Development of space database for automated building design review system. *Autom. ConStruct.* 24, 203–212.
- Lee, Y.-C., Eastman, C.M., Solihin, W., Richard, S., 2015a. Modularized rule-based validation of a BIM model pertaining to model views. *Autom. ConStruct.* 63, 1–11.
- Lee, Y.-C., Eastman, C.M., Lee, J.K., 2015b. Validations for ensuring the interoperability of data exchange of a building information model. *Autom. ConStruct.* 58, 176–195.
- Lee, H., Lee, S., Park, S., Lee, J.-K., 2015c. An approach to translate Korean Building Act into computer-readable form for automated design assessment. *Proc. Int. Symp. Autom. Robot. Constr. (ISARC)* 32.
- Lee, H., Lee, J.-K., Park, S., Kim, I., 2016. An approach to translate the Korean Building Act into a computer-executable form for evaluating building permit requirements. *Autom. ConStruct.* 71, 49–61.
- Lee, C.Y., Shim, U.J., Ahn, Y.S., 2012. A Basic Study on Review the Classification System and the Process of BIM Construction for an Automatic Review of Building Code. *Korean Journal of Construction Engineering and Management* 13 (5), 45–52.
- Lee, J.-K., Shin, J., Lee, Y., 2018a. Circulation analysis of design alternatives for elderly housing unit allocation using building information modelling-enabled indoor walkability index. *Indoor Built Environ.* 1–17.
- Lee, Y.-C., Eastman, C., Solihin, W., 2018b. Logic for ensuring the data exchange integrity of building information models. *Autom. ConStruct.* 85, 249–262.
- Lee, Y.-C., Solihin, W., Eastman, C.M., 2019. The mechanism and challenges of validating a building information model regarding data exchange standards. *Autom. ConStruct.* 100, 118–128.
- Linz, P., 2011. *An Introduction to Formal Languages and Automata*. Jones & Bartlett Publishers.
- Luo, H., Gong, P., 2015. A BIM-based code compliance checking process of deep foundation construction plans. *J. Intell. Rob. Syst.* 79, 549–576.
- Luo, H., Ling, L., Chen, K., Fordjour, A.A.M., Chen, L., 2022. Digital Technology for Quality Management in Construction: A Review and Future Research Directions. *Developments in the Built Environment*, 100087.
- Malsane, S., Matthews, J., Lockley, S., Love, P.E.D., Greenwood, D., 2015. Development of an object model for automated compliance checking. *Autom. ConStruct.* 49, 51–58.
- Mazairac, W., Beetz, J., 2013. BIMQL—An open query language for building information models. *Adv. Eng. Inf.* 27 (4), 444–456.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G.S., Dean, J., 2013. Distributed representations of words and phrases and their compositionality. *Proc. Int. Conf. Neur. Inf. Proc. Syst.* 2, 3111–3119.
- Moon, S., Lee, G., Chi, S., 2022. Automated system for construction specification review using natural language processing. *Adv. Eng. Inf.* 51, 101495.
- Nawari, N.O., 2012. BIM-model checking in building design. *Struct. Congr.* 941–952, 2012.
- Nawari, N.O., 2013. SmartCodes and BIM. In: *Structures Congress 2013: Bridging Your Passion with Your Profession*, pp. 928–937.
- Nguyen, T.H., Kim, J.L., 2011. Building code compliance checking using BIM technology. In: *Proceedings of the 2011 Winter Simulation Conference*. IEEE, pp. 3395–3400 wsc.
- Oracle, the Java Tutorials - defining method. <https://docs.oracle.com/javase/tutorial/ava/javaOO/methods.html>. (Accessed 8 December 2022).
- Park, S., Lee, H., Lee, S.-I., Shin, J., Lee, J.-K., 2015. Rule checking method-centered approach to represent building permit requirements. *Proc. Int. Symp. Autom. Robot. Constr. (ISARC)* 32.
- Parr, T., 2008. *The Definitive ANTLR Reference: Building Domain-specific Languages*. Pragmatic Bookshelf.
- Patlakas, P., Livingstone, A., Hairstans, R., Neighbour, G., 2018. Automatic code compliance with multi-dimensional data fitting in a BIM context. *Adv. Eng. Inf.* 38, 216–231.
- Peng, Y., Yan, S., Lu, Z., 2019. Transfer Learning in Biomedical Natural Language Processing: an Evaluation of BERT and ELMo on Ten Benchmarking Datasets arXiv preprint arXiv:1906.05474.
- Ramanauskaitė, S., Shein, A., Čenys, A., Rastenis, J., 2022. Security ontology structure for formalization of security document knowledge. *Electronics* 11, 1103. <https://doi.org/10.3390/electronics11071103>.
- Raslan, R., Davies, M., 2006. An analysis of industry capability for the implementation of a software-based compliance approach for the UK Building Regulations 2006. *Build. Serv. Eng. Technol.* 31, 141–162.
- Regulatory room - building smart. <http://www.buildingsmart.org/standards/standards-organization/rooms/regulatory-room/>. (Accessed 7 September 2022).
- Rehurek, R., Sojka, P., 2010. Software framework for topic modeling with large corpora. In: *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*, pp. 45–50.
- Sacks, R., Ma, L., Yosef, R., Borrmann, A., Daum, S., Kattel, U., 2017. Semantic enrichment for building information modeling: procedure for compiling inference rules and operators for complex geometry. *J. Comput. Civ. Eng.* 31 (6), 04017062.
- Seumter, B., BIM-Enabled e-submission & building act legality verification system of Korea. <https://www.eais.go.kr/>. (Accessed 2 September 2022).
- Shen, Q., Wu, S., Deng, Y., Deng, H., Cheng, J.C., 2022. BIM-based dynamic construction safety rule checking using ontology and natural language processing. *Buildings* 12 (5), 564.
- Solihin, W., Eastman, C., 2015. Classification of rules for automated BIM rule checking development. *Autom. ConStruct.* 53, 69–82.
- Specification of the high-level methods. Method Manual. <http://designitlab.kr/bim/methodManual.asp>.
- Tenney, I., Das, D., Pavlick, E., 2019. BERT Rediscovered the Classical NLP Pipeline, 05950 arXiv preprint arXiv:1905.
- Teo, A.L., Cheng, T.F., 2006. Building SMART - a strategy for implementing BIM solution in Singapore. *Synth. Met. J.* 5, 117–124.
- TIOBE, 2019. TIOBE Index for. <https://www.tiobe.com/tiobe-index>. (Accessed 22 November 2022).
- Uhm, M., Lee, G., Park, Y., Kim, S., Jung, J., Lee, J.-K., 2015. Requirements for computational rule checking of requests for proposals (RFPs) for building designs in South Korea. *Adv. Eng. Inf.* 29 (3), 602–615.
- Ventura, S.M., Getuli, V., Capone, P., Ciribini, A., 2016. Construction health and safety code checking: a BIM-based validation process. *Proc. ISTEA* 269–278.
- Wang, H.H., Boukamp, F., Elghamrawy, T., 2011. Ontology-based approach to context representation and reasoning for managing context-sensitive construction information. *J. Comput. Civ. Eng.* 25 (5), 331–346.
- Wu, J., Xue, X., Zhang, J., 2023. Invariant signature, logic reasoning, and semantic natural language processing (NLP)-Based automated building code compliance checking (I-snacc) framework. *J. Inf. Technol. Construct.* 28 (1), 1–18.
- XML, W3C. <http://www.w3.org/TR/WD-xml-961114.html>.
- Xu, X., Cai, H., 2020. Semantic approach to compliance checking of underground utilities. *Autom. ConStruct.* 109, 103006.
- Yan, H., Ma, M., Wu, Y., Fan, H., Dong, C., 2022. Overview and Analysis of the Text Mining Applications in the Construction Industry. *Heliyon*, e12088.
- Zhang, J., El-Gohary, N.M., 2013. Semantic NLP-based information extraction from construction regulatory documents for automated compliance checking. *J. Comput. Civ. Eng.* 30 (2), 04015014.
- Zhang, J., El-Gohary, N.M., 2015a. Automated information transformation for automated regulatory compliance checking in construction. *J. Comput. Civ. Eng.* 29, B4015001.
- Zhang, J., El-Gohary, N.M., 2015b. Automated extraction of information from building information models into a semantic logic-based representation. *Comput. civ. eng.* 173–180, 2015.
- Zhang, J., El-Gohary, N.M., 2016. Extending building information models semiautomatically using semantic natural language processing techniques. *J. Comput. Civ. Eng.* 30 (5), C4016004.
- Zhang, J., El-Gohary, N.M., 2017. Integrating semantic NLP and logic reasoning into a unified system for fully-automated code checking. *Autom. ConStruct.* 73, 45–57.
- Zhong, B.T., Ding, L.Y., Luo, H.B., Zhou, Y., Hu, Y.Z., Hu, H.M., 2012. Ontology-based semantic modeling of regulation constraint for automated construction quality compliance checking. *Autom. ConStruct.* 28, 58–70.
- Zhong, B., Gan, C., Luo, H., H. Xing, X., 2018. Ontology-based framework for building environmental monitoring and compliance checking under BIM environment. *Build. Environ.* 141, 127–142.
- Zhong, B., Wu, H., Li, H., Sepasgozar, S., Luo, H., He, L., 2019. A scientometric analysis and critical review of construction related research. *Autom. ConStruct.* 101, 17–31.
- Zhong, H., Xiao, C., Tu, C., Zhang, T., Liu, Z., Sun, M., 2020. How Does NLP Benefit Legal System: A Summary of Legal Artificial Intelligence arXiv preprint arXiv:2004.12158.
- Zhou, P., El-Gohary, N., 2017. Ontology-based automated information extraction from building energy conservation codes. *Autom. ConStruct.* 74, 103–117.
- Zhou, Y., She, J., Huang, Y., Li, L., Zhang, L., Zhang, J., 2022. A design for safety (DFS) semantic framework development based on natural language processing (NLP) for automated compliance checking using BIM: the case of China. *Buildings* 12 (6), 780.